Department of Computer Engineering & Applications
GLA UNIVERSITY, MATHURA

SEGMENT TREE

Problem Statements (Set 2)

Sanjay Madaan

26

Problem 6: Range XOR Query

---

1. Problem Understanding

You are given an array `nums`.

You need to support:

1. **Update**
2. `nums[i] = val`
3. **Query**
4. `find XOR of elements in range [l, r]`

Example

```
nums = [1, 3, 5]

query(0,2) → 1 ^ 3 ^ 5 = 7
update(1,2) → [1,2,5]
query(0,2) → 1 ^ 2 ^ 5 = 6
```

Why XOR is interesting?

Because XOR has **unique properties**:

```
a ^ a = 0
a ^ 0 = a
```

Helps in:

- Range queries
- Finding missing numbers
- Bit manipulation problems

---

2. Why Segment Tree?

Because XOR is:

✔ Associative
✔ Has identity element (0)

So:

```
a ^ b ^ c = (a ^ b) ^ c
```

---

3. Key Idea

Each node stores:

```
XOR of that segment
```

---

4. Tree Example

```
nums = [1, 3, 5, 7]

Tree:
        (1^3^5^7)=0
      /              \
   (1^3)=2         (5^7)=2
   /    \          /     \
  1      3        5       7
```

---

5. Important Rules

Case	Return Value
No overlap	0
Merge	XOR

---

6. Merge Operation

```
tree[node] = left ^ right;
```

---

7. Java Implementation

```
class SegmentTreeXOR {
    int[] tree;
    int n;

    public SegmentTreeXOR(int[] nums) {
        n = nums.length;
        tree = new int[4 * n];
        build(nums, 0, 0, n - 1);
    }
```

```
    private void build(int[] nums, int node, int start, int end) {
        if (start == end) {
            tree[node] = nums[start];
        } else {
            int mid = (start + end) / 2;
            build(nums, 2 * node + 1, start, mid);
            build(nums, 2 * node + 2, mid + 1, end);
            tree[node] = tree[2 * node + 1] ^ tree[2 * node + 2];
        }
    }

    public void update(int idx, int val) {
        updateUtil(0, 0, n - 1, idx, val);
    }

    private void updateUtil(int node, int start, int end, int idx, int val) {
        if (start == end) {
            tree[node] = val;
        } else {
            int mid = (start + end) / 2;
            if (idx <= mid)
                updateUtil(2 * node + 1, start, mid, idx, val);
            else
                updateUtil(2 * node + 2, mid + 1, end, idx, val);

            tree[node] = tree[2 * node + 1] ^ tree[2 * node + 2];
        }
    }

    public int query(int l, int r) {
        return queryUtil(0, 0, n - 1, l, r);
    }

    private int queryUtil(int node, int start, int end, int l, int r) {
        if (r < start || end < l)
            return 0;

        if (l <= start && end <= r)
            return tree[node];

        int mid = (start + end) / 2;
        return queryUtil(2 * node + 1, start, mid, l, r) ^
               queryUtil(2 * node + 2, mid + 1, end, l, r);
    }
}
```

---

8. Dry Run

```
nums = [1,3,5]

query(0,2) → 7
update(1,2) → [1,2,5]
query(0,2) → 6
```

---

9. Key Learning

This problem strengthens:

Bitwise thinking + segment tree

---

10. Deep Insight (Very Important)

Operation	Identity
Sum	0
Min	+∞
Max	-∞
GCD	0
XOR	0

Always define **identity correctly**

Problem 7: Maximum Subarray Sum in Range

---

1. Problem Understanding

You are given an array `nums`.

You need to support:

1. **Update**
2. `nums[i] = val`
3. **Query**
4. `find maximum subarray sum in range [l, r]`

Example

```
nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]

query(0,8) → 6
subarray = [4, -1, 2, 1]
```

Why this is HARD?

Because:

You are NOT just combining values
You are combining **subarray structures**

---

2. Why Segment Tree?

Brute force:

- Check all subarrays → $O(n^2)$ □

Segment tree:

- Combine results smartly → $O(\log n)$ □

---

3. Key Idea (MOST IMPORTANT)

Each node must store **4 values**:

```
1. sum      → total sum of segment
2. prefix   → max sum starting from left
3. suffix   → max sum ending at right
4. maxSum   → maximum subarray sum
```

---

4. Node Structure

```
class Node {
    int sum, prefix, suffix, maxSum;

    Node(int val) {
        sum = prefix = suffix = maxSum = val;
    }
}
```

---

5. Merge Logic (CORE OF PROBLEM)

If left = L and right = R:

```
sum = L.sum + R.sum

prefix = max(L.prefix, L.sum + R.prefix)

suffix = max(R.suffix, R.sum + L.suffix)

maxSum = max(
    L.maxSum,
    R.maxSum,
    L.suffix + R.prefix
)
```

This is the **heart of the problem**

---

6. Visualization

```
Left segment  | Right segment

Best subarray could be:
1. Fully in left
2. Fully in right
3. Crossing middle → (left suffix + right prefix)
```

---

7. Java Implementation

```
class SegmentTreeMaxSubarray {

    class Node {
        int sum, prefix, suffix, maxSum;

        Node(int val) {
            sum = prefix = suffix = maxSum = val;
        }
    }

    Node[] tree;
    int n;

    public SegmentTreeMaxSubarray(int[] nums) {
        n = nums.length;
        tree = new Node[4 * n];
        build(nums, 0, 0, n - 1);
    }

    private Node merge(Node left, Node right) {
        Node res = new Node(0);
        res.sum = left.sum + right.sum;
        res.prefix = Math.max(left.prefix, left.sum + right.prefix);
        res.suffix = Math.max(right.suffix, right.sum + left.suffix);
        res.maxSum = Math.max(
            Math.max(left.maxSum, right.maxSum),
            left.suffix + right.prefix
        );
        return res;
    }

    private void build(int[] nums, int node, int start, int end) {
        if (start == end) {
            tree[node] = new Node(nums[start]);
        } else {
            int mid = (start + end) / 2;
            build(nums, 2 * node + 1, start, mid);
            build(nums, 2 * node + 2, mid + 1, end);
            tree[node] = merge(tree[2 * node + 1], tree[2 * node + 2]);
        }
    }

    public void update(int idx, int val) {
        updateUtil(0, 0, n - 1, idx, val);
    }

    private void updateUtil(int node, int start, int end, int idx, int val) {
        if (start == end) {
            tree[node] = new Node(val);
        } else {
            int mid = (start + end) / 2;
            if (idx <= mid)
                updateUtil(2 * node + 1, start, mid, idx, val);
            else
```

```
                updateUtil(2 * node + 2, mid + 1, end, idx, val);

            tree[node] = merge(tree[2 * node + 1], tree[2 * node + 2]);
        }
    }

    public int query(int l, int r) {
        return queryUtil(0, 0, n - 1, l, r).maxSum;
    }

    private Node queryUtil(int node, int start, int end, int l, int r) {
        if (l == start && r == end)
            return tree[node];

        int mid = (start + end) / 2;

        if (r <= mid)
            return queryUtil(2 * node + 1, start, mid, l, r);
        else if (l > mid)
            return queryUtil(2 * node + 2, mid + 1, end, l, r);
        else {
            Node left = queryUtil(2 * node + 1, start, mid, l, mid);
            Node right = queryUtil(2 * node + 2, mid + 1, end, mid + 1, r);
            return merge(left, right);
        }
    }
}
```

---

8. Dry Run Insight

```
nums = [4, -1, 2, 1]

Max subarray = 6
```

Computed via:

```
left suffix + right prefix
```

---

9. Key Learning (CRITICAL)

This problem teaches:

Segment tree can store **multiple attributes per node**

---

10. Interview Insight

If interviewer asks:

“Can segment tree handle complex queries?”

This is your answer.

Problem 8: Range Frequency Query

---

1. Problem Understanding

You are given an array `nums`.

You need to support:

1. **Update**
2. `nums[i] = val`
3. **Query**
4. `count occurrences of number x in range [l, r]`

Example

```
nums = [1, 2, 1, 3, 1]

query(0,4,1) → 3
query(1,3,1) → 1
update(2,2) → [1,2,2,3,1]
query(0,4,1) → 2
```

Why is this challenging?

Because:

Each segment must store **multiple values**
Not just one number (like sum/min/max)

---

2. Why Segment Tree?

We want:

- Fast updates
- Fast range queries

But brute force:

- Query → O(n) □

---

3. Key Idea

Each node stores:

```
Map<Value → Frequency>
```

Example Node

For segment `[1,2,1]`:

```
{
  1 → 2
  2 → 1
}
```

---

4. Merge Logic

```
merge(leftMap, rightMap):
    resultMap = new map
    for each key:
        result[key] = left[key] + right[key]
```

Problem with this approach

Merge takes **O(n)**
Total complexity becomes **O(n log n)**

This is **NOT optimal for large constraints**

---

5. Better Approach (IMPORTANT)

Instead of segment tree:

Use **Index Mapping (HashMap + Binary Search)**

Optimized Idea

Store:

```
value → list of indices
```

Example:

```
nums = [1,2,1,3,1]

map:
1 → [0,2,4]
2 → [1]
3 → [3]
```

Query

To count occurrences of `x` in `[l, r]`:

Find indices in list using binary search

---

6. Complexity

Operation	Time
Build	O(n)
Query	O(log n)
Update	Hard (needs rebuild)

---

7. Java Implementation (Optimized Approach)

```
import java.util.*;

class RangeFreqQuery {
    Map<Integer, List<Integer>> map;

    public RangeFreqQuery(int[] arr) {
        map = new HashMap<>();
        for (int i = 0; i < arr.length; i++) {
            map.computeIfAbsent(arr[i], k -> new ArrayList<>()).add(i);
        }
    }

    public int query(int left, int right, int value) {
        if (!map.containsKey(value)) return 0;

        List<Integer> list = map.get(value);

        int l = lowerBound(list, left);
        int r = upperBound(list, right);
```

```
        return r - 1;
    }

    private int lowerBound(List<Integer> list, int target) {
        int low = 0, high = list.size();
        while (low < high) {
            int mid = (low + high) / 2;
            if (list.get(mid) >= target) high = mid;
            else low = mid + 1;
        }
        return low;
    }

    private int upperBound(List<Integer> list, int target) {
        int low = 0, high = list.size();
        while (low < high) {
            int mid = (low + high) / 2;
            if (list.get(mid) > target) high = mid;
            else low = mid + 1;
        }
        return low;
    }
}
```

---

8. Key Learning (VERY IMPORTANT)

Not every problem should use segment tree

Decision Rule

Condition	Use Segment Tree?
Single value per node	✔
Small merge cost	✔
Large maps/structures	□

---

9. Interview Insight

If you say:

“Segment tree works but is inefficient; better approach is index mapping”

That’s a **top-tier answer**

Problem 9: K-th Smallest Element in Range

---

1. Problem Understanding

You are given an array `nums`.

You need to answer:

```
query(l, r, k):
    return k-th smallest element in subarray nums[l..r]
```

Example

```
nums = [5, 2, 6, 3, 1]

query(1,3,2) → subarray = [2,6,3]
sorted = [2,3,6]
answer = 3
```

Why is this HARD?

Because:

You need **ordering inside a range**
Not just sum/min/max

Brute Force

- Extract subarray → O(n)
- Sort → O(n log n)

Total per query → □ Too slow

---

2. Why Segment Tree?

We need:

- Efficient range queries

- Preserve ordering info

---

3. Key Idea: Merge Sort Tree

Each node stores:

```
Sorted list of elements in that segment
```

Example

```
nums = [5,2,6,3]

Tree:
        [2,3,5,6]
       /         \
   [2,5]       [3,6]
   /   \       /   \
 [5]  [2]   [6]   [3]
```

---

4. Query Strategy

We do **binary search on answer**

Guess a value `x`
Count how many elements ≤ x in range `[l, r]`

If count ≥ k → move left
Else → move right

---

5. Subproblem

Count elements ≤ x in range:

Traverse segment tree
Use binary search in each node

---

6. Complexity

Operation	Time
Build	O(n log n)
Query	O($\log^2$ n)

7. Java Implementation

```
import java.util.*;

class MergeSortTree {
    List<Integer>[] tree;
    int n;

    public MergeSortTree(int[] nums) {
        n = nums.length;
        tree = new ArrayList[4 * n];
        build(nums, 0, 0, n - 1);
    }

    private void build(int[] nums, int node, int start, int end) {
        tree[node] = new ArrayList<>();
        if (start == end) {
            tree[node].add(nums[start]);
        } else {
            int mid = (start + end) / 2;
            build(nums, 2 * node + 1, start, mid);
            build(nums, 2 * node + 2, mid + 1, end);
            merge(tree[node], tree[2 * node + 1], tree[2 * node + 2]);
        }
    }

    private void merge(List<Integer> res, List<Integer> left, List<Integer>
right) {
        int i = 0, j = 0;
        while (i < left.size() && j < right.size()) {
            if (left.get(i) < right.get(j))
                res.add(left.get(i++));
            else
                res.add(right.get(j++));
        }
        while (i < left.size()) res.add(left.get(i++));
        while (j < right.size()) res.add(right.get(j++));
    }

    // count numbers <= x in range
    public int queryCount(int node, int start, int end, int l, int r, int x)
{
        if (r < start || end < l) return 0;

        if (l <= start && end <= r) {
            return upperBound(tree[node], x);
        }

        int mid = (start + end) / 2;
```

```
        return queryCount(2 * node + 1, start, mid, l, r, x) +
               queryCount(2 * node + 2, mid + 1, end, l, r, x);
    }

    private int upperBound(List<Integer> list, int x) {
        int low = 0, high = list.size();
        while (low < high) {
            int mid = (low + high) / 2;
            if (list.get(mid) <= x) low = mid + 1;
            else high = mid;
        }
        return low;
    }
}
```

---

8. How to Answer Query(l, r, k)?

```
Binary search on value range:

low = min element
high = max element

while (low < high):
    mid = (low + high)/2
    count = elements ≤ mid in [l,r]

    if count >= k:
        high = mid
    else:
        low = mid + 1
```

---

9. Key Learning (VERY IMPORTANT)

This problem teaches:

Segment tree can store **entire sorted structures**

---

10. Interview Insight

If interviewer asks:

“Can you optimize further?”

Answer:

- Use **Persistent Segment Tree**

- Time → O(log n)

---

11. Advanced Concepts Introduced

Concept	Learned
Merge sort tree	✔
Binary search on answer	✔
Order statistics	✔

Problem 10: Count Numbers Greater Than K in Range

---

1. Problem Understanding

You are given an array `nums`.

You need to answer:

```
query(l, r, k):
    count how many elements > k in nums[l..r]
```

Example

```
nums = [5, 2, 6, 3, 1]

query(1,4,3) → subarray = [2,6,3,1]

elements > 3 → [6]

answer = 1
```

Why is this important?

Because:

It is a **direct application of merge sort tree**
But simpler than k-th smallest

---

2. Why Segment Tree?

We need:

- Fast range queries
- Value-based filtering

---

3. Key Idea

Same as previous problem:

Each node stores a **sorted list**

Example

```
nums = [5,2,6,3]

Tree:
        [2,3,5,6]
       /         \
   [2,5]       [3,6]
```

---

4. Query Strategy

To count elements **greater than k**:

```
count = total elements - elements ≤ k
```

So we just need:

Count elements ≤ k (using binary search)

---

5. Key Operation

```
elementsGreater = size - upperBound(list, k);
```

---

6. Java Implementation

```
import java.util.*;

class MergeSortTreeGreater {
    List<Integer>[] tree;
    int n;

    public MergeSortTreeGreater(int[] nums) {
        n = nums.length;
        tree = new ArrayList[4 * n];
        build(nums, 0, 0, n - 1);
    }

    private void build(int[] nums, int node, int start, int end) {
        tree[node] = new ArrayList<>();
        if (start == end) {
            tree[node].add(nums[start]);
```

```
        } else {
            int mid = (start + end) / 2;
            build(nums, 2 * node + 1, start, mid);
            build(nums, 2 * node + 2, mid + 1, end);
            merge(tree[node], tree[2 * node + 1], tree[2 * node + 2]);
        }
    }

    private void merge(List<Integer> res, List<Integer> left, List<Integer>
right) {
        int i = 0, j = 0;
        while (i < left.size() && j < right.size()) {
            if (left.get(i) < right.get(j))
                res.add(left.get(i++));
            else
                res.add(right.get(j++));
        }
        while (i < left.size()) res.add(left.get(i++));
        while (j < right.size()) res.add(right.get(j++));
    }

    public int query(int node, int start, int end, int l, int r, int k) {
        if (r < start || end < l) return 0;

        if (l <= start && end <= r) {
            int countLE = upperBound(tree[node], k);
            return tree[node].size() - countLE;
        }

        int mid = (start + end) / 2;
        return query(2 * node + 1, start, mid, l, r, k) +
               query(2 * node + 2, mid + 1, end, l, r, k);
    }

    private int upperBound(List<Integer> list, int k) {
        int low = 0, high = list.size();
        while (low < high) {
            int mid = (low + high) / 2;
            if (list.get(mid) <= k) low = mid + 1;
            else high = mid;
        }
        return low;
    }
}
```

---

7. Dry Run

```
nums = [5,2,6,3]

query(0,3,3)

sorted = [2,3,5,6]
elements ≤ 3 → 2
total = 4
```

```
answer = 4 - 2 = 2
```

---

8. Key Learning

This is a **template problem** for:

- Greater than K
- Less than K
- Range counting

---

9. Interview Insight

If interviewer asks:

“Can you modify this for k-th smallest?”

You say:

- Use binary search on value (Problem 9)

---

10. Pattern Summary

Problem	Approach
K-th smallest	Binary search + count
Count > k	Direct query
Count < k	Direct query